

Un **grafo** es una estructura de datos que permite modelar relaciones entre objetos. Está compuesto por un conjunto de **nod**os (o vértices) y **aristas** (o enlaces) que conectan pares de nodos. Los grafos pueden ser dirigidos o no dirigidos, y también pueden ser ponderados si las aristas tienen un peso asociado.

Existen dos formas comunes de representar un grafo en programación:

1. **Matriz de adyacencia:** Es una matriz cuadrada donde las filas y columnas representan nodos, y los valores en la matriz indican si existe una arista entre dos nodos.
2. **Lista de adyacencia:** Es una lista o diccionario donde cada nodo tiene asociado una lista de nodos a los que está conectado.

Código en Python para representar un grafo usando ambas técnicas:

1. Matriz de adyacencia

```
# Representación del grafo usando una matriz de adyacencia
class GrafoMatrizAdyacencia:
    def __init__(self, num_vertices):
        self.num_vertices = num_vertices
        # Inicializamos una matriz num_vertices x num_vertices con ceros
        self.matriz_adyacencia = [[0] * num_vertices for _ in
range(num_vertices)]

    def agregar_arista(self, u, v):
        # Añadimos una arista entre u y v (grafo no dirigido)
        self.matriz_adyacencia[u][v] = 1
        self.matriz_adyacencia[v][u] = 1

    def mostrar_matriz(self):
        for fila in self.matriz_adyacencia:
            print(fila)
```

2. Lista de adyacencia

```
# Representación del grafo usando una lista de adyacencia
class GrafoListaAdyacencia:
    def __init__(self):
        # Usamos un diccionario para almacenar las listas de adyacencia
        self.lista_adyacencia = {}

    def agregar_vertice(self, v):
        if v not in self.lista_adyacencia:
            self.lista_adyacencia[v] = []

    def agregar_arista(self, u, v):
```

```

        # Añadimos una arista entre u y v (grafo no dirigido)
        if u in self.lista_adyacencia and v in self.lista_adyacencia:
            self.lista_adyacencia[u].append(v)
            self.lista_adyacencia[v].append(u)

    def mostrar_lista(self):
        for vertice in self.lista_adyacencia:
            print(f"{vertice}: {self.lista_adyacencia[vertice]}")

```

**** Ejemplos de cada tipo ****

. Grafo dirigido con peso usando matriz de adyacencia

```

# Representación del grafo dirigido con peso usando una matriz de adyacencia
class GrafoDirigidoConPesoMatriz:
    def __init__(self, num_vertices):
        self.num_vertices = num_vertices
        self.matriz_adyacencia = [[0] * num_vertices for _ in
range(num_vertices)]

    def agregar_arista(self, u, v, peso):
        self.matriz_adyacencia[u][v] = peso

    def mostrar_matriz(self):
        for fila in self.matriz_adyacencia:
            print(fila)

```

. Grafo dirigido con peso usando lista de adyacencia

```

# Representación del grafo dirigido con peso usando una lista de adyacencia
class GrafoDirigidoConPesoLista:
    def __init__(self):
        self.lista_adyacencia = {}

    def agregar_vertice(self, v):
        if v not in self.lista_adyacencia:
            self.lista_adyacencia[v] = []

    def agregar_arista(self, u, v, peso):
        if u in self.lista_adyacencia:
            self.lista_adyacencia[u].append((v, peso))

    def mostrar_lista(self):
        for vertice in self.lista_adyacencia:
            print(f"{vertice}: {self.lista_adyacencia[vertice]}")

```

. Grafo dirigido sin peso usando matriz de adyacencia

```
# Representación del grafo dirigido sin peso usando una matriz de adyacencia
class GrafoDirigidoSinPesoMatriz:
    def __init__(self, num_vertices):
        self.num_vertices = num_vertices
        self.matriz_adyacencia = [[0] * num_vertices for _ in
range(num_vertices)]

    def agregar_arista(self, u, v):
        self.matriz_adyacencia[u][v] = 1

    def mostrar_matriz(self):
        for fila in self.matriz_adyacencia:
            print(fila)
```

. Grafo dirigido sin peso usando lista de adyacencia

```
# Representación del grafo dirigido sin peso usando una lista de adyacencia
class GrafoDirigidoSinPesoLista:
    def __init__(self):
        self.lista_adyacencia = {}

    def agregar_vertice(self, v):
        if v not in self.lista_adyacencia:
            self.lista_adyacencia[v] = []

    def agregar_arista(self, u, v):
        if u in self.lista_adyacencia:
            self.lista_adyacencia[u].append(v)

    def mostrar_lista(self):
        for vertice in self.lista_adyacencia:
            print(f"{vertice}: {self.lista_adyacencia[vertice]}")
```

. Grafo no dirigido con peso usando matriz de adyacencia

```
# Representación del grafo no dirigido con peso usando una matriz de adyacencia
class GrafoNoDirigidoConPesoMatriz:
    def __init__(self, num_vertices):
        self.num_vertices = num_vertices
        self.matriz_adyacencia = [[0] * num_vertices for _ in
range(num_vertices)]

    def agregar_arista(self, u, v, peso):
        self.matriz_adyacencia[u][v] = peso
```

```

        self.matriz_adyacencia[v][u] = peso

    def mostrar_matriz(self):
        for fila in self.matriz_adyacencia:
            print(fila)

```

. Grafo no dirigido con peso usando lista de adyacencia

```

# Representación del grafo no dirigido con peso usando una lista de adyacencia
class GrafoNoDirigidoConPesoLista:
    def __init__(self):
        self.lista_adyacencia = {}

    def agregar_vertice(self, v):
        if v not in self.lista_adyacencia:
            self.lista_adyacencia[v] = []

    def agregar_arista(self, u, v, peso):
        if u in self.lista_adyacencia and v in self.lista_adyacencia:
            self.lista_adyacencia[u].append((v, peso))
            self.lista_adyacencia[v].append((u, peso))

    def mostrar_lista(self):
        for vertice in self.lista_adyacencia:
            print(f"{vertice}: {self.lista_adyacencia[vertice]}")

```

. Grafo no dirigido sin peso usando matriz de adyacencia

```

# Representación del grafo no dirigido sin peso usando una matriz de adyacencia
class GrafoNoDirigidoSinPesoMatriz:
    def __init__(self, num_vertices):
        self.num_vertices = num_vertices
        self.matriz_adyacencia = [[0] * num_vertices for _ in
range(num_vertices)]

    def agregar_arista(self, u, v):
        self.matriz_adyacencia[u][v] = 1
        self.matriz_adyacencia[v][u] = 1

    def mostrar_matriz(self):
        for fila in self.matriz_adyacencia:
            print(fila)

```

. Grafo no dirigido sin peso usando lista de adyacencia

```
# Representación del grafo no dirigido sin peso usando una lista de adyacencia
class GrafoNoDirigidoSinPesoLista:
    def __init__(self):
        self.lista_adyacencia = {}

    def agregar_vertice(self, v):
        if v not in self.lista_adyacencia:
            self.lista_adyacencia[v] = []

    def agregar_arista(self, u, v):
        if u in self.lista_adyacencia and v in self.lista_adyacencia:
            self.lista_adyacencia[u].append(v)
            self.lista_adyacencia[v].append(u)

    def mostrar_lista(self):
        for vertice in self.lista_adyacencia:
            print(f"{vertice}: {self.lista_adyacencia[vertice]}")
```